창의융합과제연구 보고서

Shot-Efficient Calibration of a Single-Qubit Rotation Gate: A Comparison of Classical Optimizers Under Measurement Noise

2026 년 7 월

용인한국외국어대학교부설고등학교

은 석 준 (David Eun)

Shot-Efficient Calibration of a Single-Qubit Rotation Gate: A Comparison of Classical Optimizers Under Measurement Noise

지도교사 윤태선 인

이 보고서를 HAFS 창의융합과제연구보고서로 제출함.

2026년 7월 13일

용인한국외국어대학교부설고등학교

은 석 준

은석준의 창의융합과제연구 보고서를 인준함.

2026년 7월 13일

교 장 정영우 인

교 감 박인호 인

연구부장 윤태선 인

Abstract

Shot-Efficient Calibration of a Single-Qubit Rotation Gate: A Comparison of Classical Optimizers Under Measurement Noise

David Eun

International Track

Hankuk Academy of Foreign Studies

Calibration of the rotation angle of a single-qubit RY gate can be viewed as a noisy black-box optimization problem at its core, since the calibrator cannot directly access the gate's true results (parameters) except for a stochastic estimation. This paper evaluates in simulation with shot-noise and compares four optimization strategies for recovering an unknown $R_y(\theta)$ rotation angle to within 1° accuracy: Grid search, Nelder-Mead, COBYLA, and SPSA, using shot count as the primary cost metric. In ideal simulations and excluding grid search, SPSA had the highest convergence rate (~80%) and the highest average cost. Nelder-Mead had the lowest convergence rate (~27%) and the lowest average cost when it did converge. However, the central finding and argument of this work is the importance of considering *silent convergence failures* under shot noise, frequently seen in Nelder-Mead and COBYLA, and its implications in designing calibration pipelines for real quantum hardware, in which the ground truth is unavailable, alongside the scope and limitations of a single-parameter, simulation-based study.

Keywords: Quantum gate calibration, Classical optimizers, Shot-noise, Shot efficiency, Silent convergence failure, Black-box optimization.

Student Number: 30109

Table of Contents

1. Introduction.....5

2. Background.....6

2.1 Single-Qubit States and the RY gate.....6

2.2 The Born Rule and Measurement Probabilities.....6

3. Methodology.....6

3.1 Problem Formulation.....6

3.2 Baseline: Grid Search.....7

3.3 Classical Optimizers.....7

3.4 Two Notions of Convergence.....8

3.5 Experimental Protocol.....9

3.6 Realistic Validation via Simulated Hardware Backends.....9

3.7 Scope.....10

4. Results.....10

4.1 Shot Efficiency and Convergence Reliability.....10

4.2 The Declared-vs-Graded Convergence Gap.....11

4.3 Robustness under Device-Calibrated Noise.....12

4.4 Summary.....13

5. Discussion.....14

5.1 Shot efficiency was never the whole question.....14

5.2 Why COBYLA and SPSA diverge.....15

5.3 What the VQE literature's rationale does and doesn't transfer.....16

5.4 Robustness under noise: one confident finding, one open question.....16

5.5 Scope of the claim.....17

6. Limitations.....18

7. Future Work.....20

8. Conclusion.....21

References.....22

국문 초록.....23

1. Introduction

As quantum computers develop, calibration — the process of tuning and optimizing a quantum computer's control parameters to maintain the accuracy and performance of quantum operations under changes in the hardware and the environment — is becoming more important, especially since every quantum circuit that executes on quantum computers depends on calibrated gates.

For a single-qubit rotation gate, such as $R_y(\theta)$, calibration means *empirically* finding the values of the control parameters (typically pulse amplitude and duration, abstracted to an angle θ) that produce a target quantum state or target measurement statistics.

Note the word "empirically." During the process, the direct inspection of θ is not possible. The θ that results from the system can only be inferred through repeated measurement, each of which is a single-shot, probabilistic query. Therefore, the experimentalist or the optimizer can only ever see noisy estimates of the probability.

This study grew out of an internship experience at a quantum computing lab, where I developed programs and QCoDeS drivers for controlling quantum computers. Much of that work sat adjacent to calibration routines and later prompted the question that this paper investigates: How much does the choice of classical optimizer affect how many measurements calibration takes, given that every measurement has a cost in shots, and can an optimizer's stopping decision be trusted?

To try and find an answer to that question, this paper simplifies single-qubit rotation gate calibration to a one-dimensional noisy black-box optimization problem.

The following are considered:

1. **Shot Efficiency**: When a fixed accuracy target is given (e.g. recover θ to within 1°), how many oracle[1)] calls does each optimizer require, on average, to reach that accuracy?
2. **Convergence Reliability**: When an optimizer claims that it has converged, how often is the declaration actually correct?

And four methods are compared: Grid search, Nelder-Mead, COBYLA, and SPSA.

1) An operation that gives access to a black box function, which allows quantum algorithms to query it without knowing its internals.

2. Background

This section briefly reviews the background knowledge required to follow the rest of the paper.

2.1 Single-Qubit States and the RY gate

A pure state of a single qubit can be represented as a superposition of the computational basis states $|0\rangle$ and $|1\rangle$. If $\alpha, \beta \in \mathbb{C}$ and $|\alpha|^2 + |\beta|^2 = 1$,

$$|\psi\rangle = \alpha\,|0\rangle + \beta|1\rangle$$

The $R_y\,(\theta)$ gate is the operation of rotating a qubit around the Y-axis of the Bloch sphere by a specified angle θ. Applying the RY gate to an initial state of $|0\rangle$ results in the superposition

$$R_y\,(\theta) = \cos\left(\frac{\theta}{2}\right)|0\rangle + \sin\left(\frac{\theta}{2}\right)|1\rangle$$

Physically, θ corresponds to multiple control parameters of the device that implements the gate (for example, the amplitude or duration of a microwave pulse on a superconducting qubit).

2.2 The Born Rule and Measurement Probabilities

The measurement of a qubit is probabilistic. The probability of measuring the qubit in superposition as above in the state $|1\rangle$ is given by the Born rule:

$$P(1) = |\langle 1|\psi\rangle|^2 = \sin^2\left(\frac{\theta}{2}\right)$$

3. Methodology

3.1 Problem Formulation

We consider the task of recovering the rotation angle $\theta \in [0, \pi]$ applied by a single-qubit RY gate from noisy measurement statistics. This reflects the real structure faced when calibrating a physical single-qubit gate, in which the true rotation angle implemented by a control pulse *cannot* be known a priori and must be empirically inferred from repeated measurement.

Given a qubit initialized to $|0\rangle$, the circuit applies $R_y(\theta)$ and measures it in the computational basis. By the Born rule, the probability of measuring $|1\rangle$ is

$$p(\theta) = \sin^2\left(\frac{\theta}{2}\right)$$

However, because $p(\theta)$ can only be estimated by repeated sampling, we only have access to an empirical estimate $\hat{p}(\theta) = k/N$, where k is the number of $|1\rangle$ outcomes observed over N shots. Gate calibration can be seen as a **noisy black-box optimization problem**: each query to the oracle costs shots and returns a stochastic estimate instead of the true value.

Each optimizer receives the same *p_target* value, which is the theoretical expectation value calculated using $p(\theta)$, and minimizes the cost function,

$$C(\theta_{\text{guess}}) = (\hat{p}(\theta_{\text{guess}}) - p_{\text{target}})^2$$

using its own strategy. Every call to C consumes *SHOTS_PER_EVAL = 100* shots via the oracle.

3.2 Baseline: Grid Search

Grid search is a brute-force optimization method that sweeps the parameter across a fixed set of evenly spaced points within the range of possible values.

For the oracle, it evaluates it at *GRID_POINTS = 180* evenly spaced points over $[0, \pi]$ — a fixed 1° angular resolution. The algorithm allocates *shots_per_point = total_shots // n_points* shots to each point and returns the point with the lowest cost.

Its total shot cost, *GRID_POINTS * SHOTS_PER_EVAL*, is fixed and independent of θ_{true}. As it does not adapt to information gathered during the search, it serves a useful floor (any method with adaptive behavior that uses more than grid search's shot cost while matching its reliability has no practical advantage).

3.3 Classical Optimizers

Three classical optimizers that adapt during search according to the data were benchmarked against the grid search baseline. Each used the same cost function $C(\theta_{\text{guess}})$ and the same per-evaluation shot budget (100 shots/query).

1. **Nelder-Mead**: A method that reflects, expands, or shrinks a simplex, which is a shape of $n + 1$ vertices in n dimensions, toward lower cost to find the minimum or maximum of an objective function in a multidimensional space without derivatives. It was included as a standard general-purpose, derivative-free classical optimizer.
2. **COBYLA (Constrained Optimization BY Linear Approximation)**: A derivative-free optimization algorithm that repeatedly builds a local approximate linear model of the cost function and steps toward its predicted minimum inside a shrinking trust region. It is popular and widely used in quantum computing.
3. **SPSA (Simultaneous Perturbation Stochastic Approximation)**: An optimization algorithm that estimates the gradient from two randomly-perturbed evaluations per step and performs gradient descent. Its design explicitly states that it is suitable for optimization of noisy objective functions.

The implementation of *scipy.optimize* was used for the first two and *qiskit_algorithms.optimizers* for SPSA. All three optimizers were run with near-default settings rather than hand-tuned configurations.

Optimizer	Key Settings
Nelder-Mead	*maxiter=200*, *xatol=1e-3* rad (≈0.057°), *fatol=1e-6*, bounds [0, π]
COBYLA	*maxiter=200*, *tol=1e-6*, linear inequality constraints for [0, π]
SPSA	*maxiter=100*

Table 1. The key parameter settings of each optimizer.

3.4 Two Notions of Convergence

A central methodological distinction in this study is between two different senses in which an optimizer can be said to have "converged":

- **Self-declared convergence**: The point at which the internal stopping criteria of the optimizer fires or its iteration budget is exhausted.
- **Benchmark-graded convergence**: The point at which the optimizer's running best estimate first comes within 1° of θ_{true} and remains within that boundary thereafter. This is also the fixed accuracy target mentioned before.

This distinction makes it possible to detect cases where an optimizer halts while its final estimate is still more than 1° from θ_{true}.

In the code used for the experiment, *TrackedOracle* wraps calls to the oracle to record cumulative shot usage; because it has access to θ_{true}, to grade each evaluation against the 1° criterion independently of what the optimizer believes about its own progress; and to report the number of shots that was needed for the optimizer to reach benchmark-graded convergence, if it ever did.

3.5 Experimental Protocol

Each calibration method was evaluated over *N_TRIALS = 15* randomly sampled true angles for averaged and defensible results. They were drawn once from a master RNG seeded with *SEED = 42* before trials and reused across all three optimizers.

Also, for each trial index i, a fresh RNG was deterministically seeded as *SEED * 1000 + i* and reused across all methods at that same index.

All results are reported as mean, standard deviation, and median shots-to-converge across the trials on which a method converged, along with the non-convergence rate.

3.6 Realistic Validation via Simulated Hardware Backends

The original plan called for validating results on live IBM Quantum hardware. However, registration for IBM Cloud access could not be completed for unknown reasons.

Hence, this step was substituted with a simulator-based approximation using *qiskit_ibm_runtime.fake_provider backends*, which mimics the behaviors of IBM Quantum systems by applying data snapshotted from a real system to a local *AerSimulator* run.

This is explicitly not equivalent to live hardware validation, and is referred to as noise-model-based validation in this paper. Yet, it still allows us to answer whether the optimizer ranking established under idealized shot noise is preserved under a general noise profile.

3.7 Scope

This study addresses the recovery of a single scalar rotation angle parameter under an ideal environment, motivated by, but simplified relative to, full physical pulse calibration, which usually involves multiple correlated parameters and time-varying device drift.

The single-parameter setting is treated here as the minimal case in which the shot-efficiency question can be asked.

4. Results

All results come from the *AerSimulator* unless stated otherwise.

4.1 Shot Efficiency and Convergence Reliability

Figure 1 summarizes the core comparison. Panel A shows the shot cost of each adaptive method, counted only over the trials that actually reached 1° accuracy at some point; Panel B shows what fraction of trials reached that threshold.

Figure 1. Shot efficiency and convergence reliability of four calibration strategies

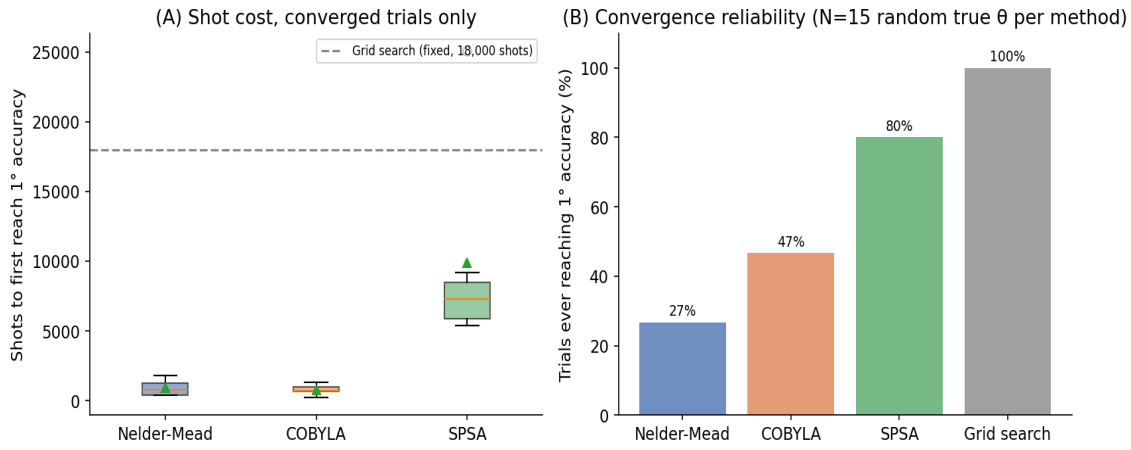


Figure 1. (A) Shots consumed before first reaching 1° *accuracy, converged trials only, compared against grid search's fixed cost. (B) Fraction of trials whose trajectory ever came within* 1° *of the true angle.*

The reliability is ordered clearly. Grid search converges in 100% of trials by design, SPSA reaches 1° accuracy at some point in 80% of trials (12/15), COBYLA in 47% (7/15), and Nelder-Mead in 27% (4/15).

The shot-cost ordering is roughly inverted: when Nelder-Mead and COBYLA do get close, they typically do so cheaply (median 750 and 700 shots respectively, versus grid search's fixed 18,000), while SPSA needs a median of 7,300 shots, which is still less than half of grid search but far more than the other two adaptive methods.

4.2 The Declared-vs-Graded Convergence Gap

For every trial, whether the optimizer declared convergence and whether the angle the optimizer returned was, in fact, within 1° of the true angle were recorded.

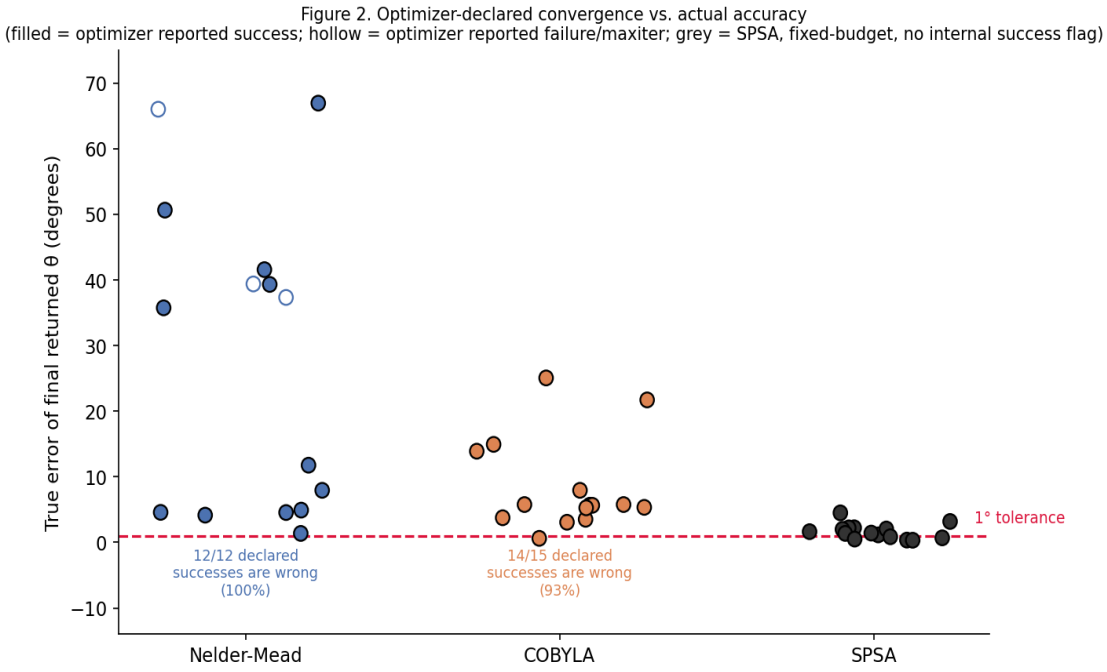


Figure 2. Each point stands for one trial. Solid = declared success; Hollow = optimizer exhausted its iteration budget without declaring success. Grey = SPSA (no internal success/failure flag; always runs its full fixed budget). The dashed line marks the 1° grading threshold.

As it can be seen in the figure, there is a stark gap. COBYLA reported success in all 15 trials, but the angle it actually returned was within 1° of true θ in only 1 of those 15 (7%). 14 of its 15 declared successes (93%) were silent failures, with returned errors as large as 25–22°. Nelder-

Mead reported success in 12 of 15 trials, but every one of those 12 declared successes was wrong by more than 1° (100% silent failure rate among declared successes), with errors up to 67° in the worst case. In both cases, an experimentalist relying on the optimizer's success flag when calibrating real hardware would have no way to know that it failed.

Meanwhile, because SPSA always spends its full fixed iteration budget and has no internal tolerance-based stopping test, although its final answers were within 1° in only 5 of 15 trials, 33%, it hints at possible imperfection rather than confidently declaring a wrong answer.

4.3 Robustness under Device-Calibrated Noise

Because live IBM Cloud queue access could not be obtained in the project timeline (see Methods), hardware validation was carried out instead using qiskit_ibm_runtime.fake_provider.FakeManilaV2, which supplies AerSimulator with a frozen snapshot of a real 5-qubit device's calibration data (T1/T2, readout error, native gate error rates). This is explicitly a noise-model simulation, not a live-hardware run; FakeManilaV2 was used in place of a larger backend such as FakeSherbrooke purely because its smaller transpilation and simulation cost made hundreds of oracle calls per optimizer run tractable within the available compute time. Given this cost, only the two better-performing methods from Section 4.1–4.2 (COBYLA and SPSA) were re-run, at a reduced sample size of 8 trials each rather than 15.

Figure 3. Effect of device-calibrated noise (FakeManilaV2) vs. ideal shot noise

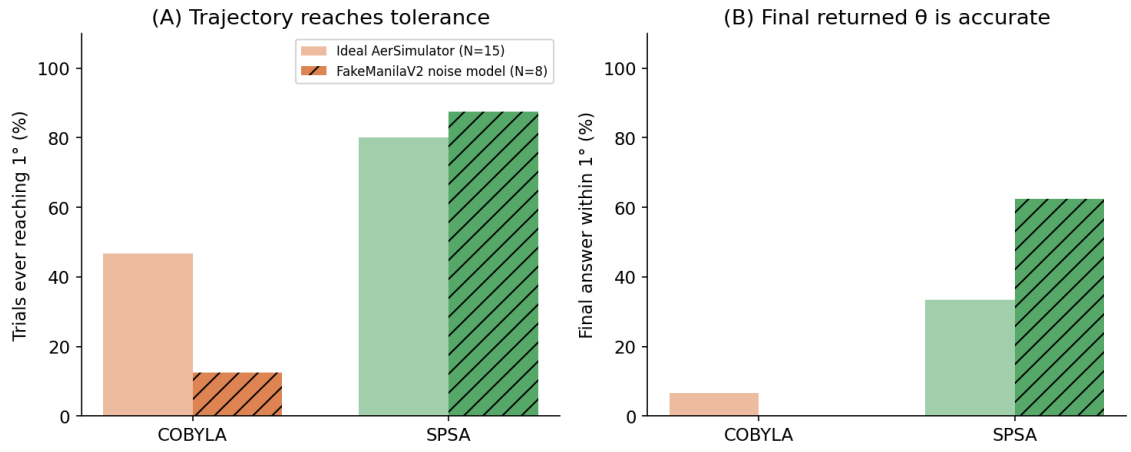


Figure 3. Convergence rate under ideal AerSimulator shot noise (N=15, from Sections 4.1–4.2) versus under FakeManilaV2 device-calibrated noise (N=8). (A) trajectory ever reaches 1°*; (B) final returned θ is within* 1°.

The ranking established in Sections 4.1–4.2 holds under device noise and, on this small sample, sharpens: COBYLA's already-poor final-answer accuracy fell further, from 7% (1/15) under ideal shot noise to 0% (0/8) under FakeManilaV2 noise, and the fraction of trials whose trajectory ever reached 1° dropped from 47% to 12.5% (1/8). SPSA, by contrast, was not degraded by the added device noise on this sample — its trajectory-reaches-threshold rate rose slightly (80% to 87.5%) and its final-answer accuracy rose from 33% (5/15) to 62.5% (5/8). This is consistent with the qualitative expectation that COBYLA, a deterministic trust-region method not designed for stochastic objectives, degrades under additional noise sources, while SPSA's gradient estimator is explicitly built to tolerate noisy measurements. That said, the apparent SPSA improvement should not be over-read: with only 8 noisy trials versus 15 ideal ones, a single trial changes the reported percentage by 12.5 points, and the two sample sizes are not matched. The honest reading of Figure 3 is that COBYLA's fragility under noise is a reasonably confident finding — it moves in the same direction as its already-weak ideal-case performance and by a large margin — while the SPSA comparison is at best suggestive and would need a matched, larger-N re-run to support a real claim of noise robustness.

4.4 Summary

Method	Ever reaches 1°	Final answer correct	Declared success but wrong	Median shots (converged)
Grid search	100% (15/15)	100% (15/15)	n/a	18,000 (fixed)
Nelder-Mead	27% (4/15)	0% (0/15)	12/12 (100%)	750
COBYLA	47% (7/15)	7% (1/15)	14/15 (93%)	700
SPSA	80% (12/15)	33% (5/15)	n/a (no flag)	7,300

Table 2.

Read together, Figure 1 and Figure 2 tell different stories about the same three optimizers. By the shot-efficiency framing that motivates using SPSA or COBYLA over grid search in the first

place (Introduction; also Peruzzo et al. 2014's rationale for Nelder-Mead in VQE), all three adaptive methods look attractive relative to grid search's fixed 18,000-shot cost. By the declared-vs-graded framing in Section 4.2, only SPSA's cost premium buys something a real calibration pipeline could actually trust: an absence of confidently wrong answers. Section 4.3's noise-model results, though limited by small sample size, are consistent with that picture holding — or worsening for COBYLA — under realistic device noise.

5. Discussion

The Results section established two separate claims: (1) SPSA, COBYLA, and Nelder-Mead can all reach a target calibration accuracy using far fewer shots than grid search, and (2) two of those three methods will frequently tell you that they succeeded when they, in fact, did not. This section expands on the meaning of them in detail.

5.1 Shot efficiency was never the whole question

Figure 1 alone would support a fairly conventional conclusion: adaptive optimizers beat grid search on shot budget, so a calibration pipeline should prefer them. That conclusion is consistent with why SPSA and COBYLA are used in Qiskit's own VQE routines and in real gate-calibration pipelines in the first place — shot budget is a real, expensive constraint, and grid search's fixed 18,000-shot cost does not scale gracefully as accuracy requirements tighten or as the number of parameters grows.

Figure 2 complicates that conclusion without overturning it. The problem is not that Nelder-Mead and COBYLA are slow or fail often — COBYLA in particular is admirably cheap when it does terminate. The problem is that both methods terminate confidently and incorrectly far more often than they terminate confidently and correctly, and there is no signal available to a real experimentalist — someone who does not have access to θ_{true} — that would distinguish a trustworthy termination from an untrustworthy one. A shot-count comparison that stops at Figure 1 would recommend COBYLA as the shot-cheapest reliable option; a comparison that includes Figure 2 has to describe COBYLA's cheapness as, in this experiment, mostly the cost of being wrong quickly.

This is the paper's central claim, and it is worth being precise about its scope: the claim is not that COBYLA or Nelder-Mead are bad optimizers in general, or that their scipy implementations are flawed. It is that their internal stopping criteria — designed for smooth, deterministic objectives — do not have a principled way to distinguish “the objective stopped changing because we found the minimum” from “the objective stopped changing because shot noise is currently masking further improvement.” Under a noisy objective, both look identical to the stopping test.

5.2 Why COBYLA and SPSA diverge

The asymmetry in Figure 2 — COBYLA declares success constantly and is usually wrong, SPSA never declares success at all and is sometimes right — follows fairly directly from how the two methods are built, not from an arbitrary implementation choice. COBYLA is a deterministic trust-region method: it fits a linear model to a handful of nearby function evaluations and shrinks its trust region when the fitted model's predictions stop improving on the observed (noisy) cost. Once the trust region is small enough that the shot noise floor dominates the signal, the fitted model can look flat by chance, and COBYLA reports success — not because it found the minimum, but because it can no longer see past the noise. Nelder-Mead's simplex shrink step has the analogous weakness: fatol and xatol are absolute tolerances with no sample-size correction, so they cannot tell a converged simplex from a simplex that has simply stopped resolving differences smaller than the noise.

SPSA sidesteps this specific failure mode by construction rather than by being more accurate: the qiskit_algorithms implementation used here runs a fixed number of iterations with a predetermined gain-decay schedule and has no tolerance-based early-exit test at all. It cannot mistake noise for convergence because it never checks. That is a real advantage for trustworthiness — nothing it reports can be a false positive — but it comes at the cost shown in Figure 1: SPSA's shot budget is fixed and comparatively large regardless of how easy or hard a given trial turns out to be, where COBYLA and Nelder-Mead can in principle finish early on easy trials. SPSA also does not achieve perfect accuracy either (67% of its final answers were still off by more than 1°); it is best read not as “the accurate method” but as “the method whose

imperfection is at least legible," since a practitioner always knows exactly how many shots were spent and gets no false assurance either way.

5.3 What the VQE literature's rationale does and doesn't transfer

SPSA and COBYLA are used in VQE not because VQE and gate calibration are the same physical problem, but because they share the same computational shape: both are noisy, black-box optimizations over a small number of continuous parameters, where each function evaluation costs real shots on real (or simulated) hardware. That shared shape is the entire basis for importing VQE's optimizer choices into a calibration context, and it is worth stating plainly that this justification is structural, not physical — nothing about SPSA's gain-decay schedule or COBYLA's trust region encodes anything about qubits, gates, or rotations. Anything that behaves like a noisy scalar-valued black-box function of a few real parameters is fair game for these methods, whether or not it has anything to do with quantum computing.

What is specific to this experiment, and does not automatically transfer from the VQE literature, is the declared-vs-graded convergence gap itself. VQE papers benchmark shot efficiency against known ground-state energies in simulation, essentially the same grading setup used in Section 4.2 — but the convergence claims in that literature are typically about the accuracy of the final energy estimate, not about whether the optimizer's own internal stopping flag can be trusted. That is a narrower, calibration-specific question, and one that matters more in a calibration context than in a VQE context: a VQE result that silently terminated early still produces an energy estimate whose error can often be bounded or checked against known physics; a calibration routine that silently terminates early produces a gate that will be used, unflagged, in every subsequent circuit that calls it.

5.4 Robustness under noise: one confident finding, one open question

The FakeManilaV2 results in Section 4.3 offer two findings of very different strength. COBYLA's further degradation under device-calibrated noise — its final-answer accuracy dropping from an already-weak 7% to 0% — moves in the direction the trust-region argument above predicts, and does so by a large enough margin relative to its ideal-case baseline that it is reasonable to treat as a real effect rather than noise in the estimate itself. SPSA's apparent

improvement under the same added noise is not treated with the same confidence in this draft: it rests on 8 trials against a 15-trial ideal-case baseline, and a single trial's outcome moves the reported percentage by more than 12 points at that sample size. The more defensible reading is that SPSA's accuracy did not clearly degrade under device noise, which is itself a useful negative result, rather than that it clearly improved.

It is also worth being explicit that Section 4.3 validates a noise model, not hardware. FakeManilaV2 supplies a frozen snapshot of one real device's T1/T2, readout error, and gate error rates to a local simulator; it does not carry any of the drift, crosstalk, or queue-time variation that a live hardware run would. The honest claim these results support is narrower than "this holds on hardware": it is "this ranking is not an artifact of the idealized, textbook shot-noise model used in Sections 4.1–4.2, and persists under at least one realistic device noise profile." Whether it holds under live queue conditions is left as future work rather than claimed here.

5.5 Scope of the claim

These results describe single-parameter calibration of one RY(θ) rotation, graded against a fixed 1° tolerance, using scipy's default Nelder-Mead and COBYLA implementations and qiskit_algorithms's default-calibrated SPSA, over 15 (or, for the noise-model comparison, 8) randomly drawn true angles. It is not a claim about multi-parameter calibration, about other gate types, about other optimizer hyperparameter settings, or about optimizers not tested here. The paper deliberately does not claim that this silent-failure pattern is a novel discovery about Nelder-Mead or COBYLA in the abstract — both methods' sensitivity to objective noise is documented in the optimization literature. The contribution here is narrower and more concrete: quantifying how often that documented sensitivity produces a specific, practically dangerous outcome — a declared-successful single-qubit gate calibration that is, in fact, wrong — under conditions modeled on how such a routine would actually be run. A fuller discussion of what would need to change for these numbers to generalize (multi-parameter targets, alternative stopping criteria, live hardware) is left to the Limitations and Future Work sections.

6. Limitations

This section is deliberately kept separate from the Discussion, rather than folded into it, so that the boundaries of the claim are easy to find and cannot be missed by a reader skimming for them.

Single parameter. The entire study calibrates one scalar angle, θ. Real gate calibration is frequently multi-parameter (amplitude and phase, or amplitude and duration, for example), and SPSA's design advantage — a per-step gradient estimate whose cost does not scale with the number of parameters — is specifically a high-dimensional advantage that this one-dimensional study cannot demonstrate. The shot-cost gap between SPSA and the other methods reported here should be read as a lower bound on SPSA's relative advantage in realistic, higher-dimensional calibration, not as the whole story.

Simulation only, no live hardware. The original plan for this project included validation on real IBM Quantum hardware. Registration for IBM Cloud access could not be completed in time, and the hardware section was replaced with noise-model-based validation using qiskit_ibm_runtime.fake_provider backends (e.g., FakeSherbrooke), which apply snapshotted real-device calibration data — T1/T2 times, readout error, gate error — to a local AerSimulator. This is a meaningfully weaker claim than a live hardware run: it captures a recorded noise profile from a real device at a past point in time, replayed deterministically, rather than the live, time-varying, and correlated noise (drift, crosstalk, temperature-dependent effects) present on an actual running system. Any claim in this paper about hardware-realistic noise should be read with that distinction in mind; "validated against a real-device noise model" is not the same claim as "validated on real hardware," and this paper does not make the latter claim.

Idealized noise model. Even the noise-model-based validation only captures the error channels the fake-backend snapshot encodes. It does not capture calibration drift during the measurement campaign itself, or state-preparation errors beyond what the model includes. In a real running experiment, the "true" θ can itself drift over the course of a calibration run, an effect entirely absent from this simulation, where true_theta is fixed by construction.

Grading requires ground truth the optimizer never has. The convergence-reliability result in Section 4 — that Nelder-Mead and COBYLA often declare convergence while still wrong by more than 1° — is only observable because the simulation retains true_theta for grading purposes, external to the optimizer's cost function. This is the intended design of the experiment, not an oversight, but it means the paper cannot report an actual rate of silent convergence failure on hardware; it can only demonstrate that the mechanism exists and quantify it under one particular noise level and shot budget. A different shot-per-evaluation setting, or a different noise floor, would shift these rates, likely substantially.

Hyperparameter fairness is a judgment call, not a solved problem. Nelder-Mead and COBYLA were run with scipy.optimize defaults (adjusted only where clearly necessary, e.g., xatol set near the noise floor rather than left at a value too tight to ever trigger). SPSA was run with qiskit_algorithms's built-in calibration phase, treated as the fairness-consistent counterpart to the scipy defaults, with those calibration-phase oracle calls included in the reported shot cost. This is a reasonable convention, but it is a convention: a differently tuned Nelder-Mead or COBYLA (for instance, using a noise-aware restart strategy) could plausibly close some of the reliability gap reported here. The claim of this paper is about these specific, defensibly-chosen configurations, not about the absolute ceiling of what each algorithm family can achieve.

Cost function shape. The cost function used throughout, $\left(\hat{P}(1) - P^*(1)\right)^2$, is one reasonable choice among several (e.g., a likelihood-based cost could be used instead) and its particular shape — including how it behaves near the target versus far from it — likely interacts with each optimizer's stopping logic in ways not separately isolated here.

Sample size. Fifteen trials per method, while enough to see a clear and consistent pattern across methods, is small by the standards of a fully powered statistical comparison. The reported convergence rates (80% / 47% / 27%) should be read as point estimates with real sampling uncertainty, not as precise figures.

7. Future Work

The most direct extension of this work is the two-parameter case floated in the original project plan: calibrating both an angle and a phase simultaneously. This is the natural next step for two reasons. First, it is the setting in which SPSA's design advantage is expected to become most visible, since its per-iteration cost is independent of parameter count while grid search's cost grows combinatorially and Nelder-Mead/COBYLA's cost grows at best linearly with dimension. Second, and more important given this paper's central finding, it would let us test whether the silent convergence failure pattern observed in one dimension gets worse, stays the same, or improves in higher dimensions — a genuinely open question, since a higher-dimensional simplex or trust region gives an optimizer more ways to be fooled by noise, but also more internal structure that could in principle be used to detect it.

A second natural extension is a noise-aware or budget-adaptive stopping criterion: rather than treating Nelder-Mead's and COBYLA's default stopping logic as fixed, one could investigate simple modifications — for instance, requiring a stopping condition to hold across several re-measurements before accepting it, at the cost of additional shots — and ask whether a small, quantifiable increase in shot budget can close most of the reliability gap identified in this paper. If so, that would reframe the paper's central finding constructively: not "avoid Nelder-Mead/COBYLA for calibration," but "pair them with a cheap noise-aware convergence check."

Third, given that live hardware access could not be obtained for this iteration of the project, a natural and comparatively low-cost follow-up is simply to complete the originally planned hardware validation once IBM Cloud access (or access to another provider) is available — even a handful of trials on real hardware, run only for the one or two best-performing methods, would upgrade the paper's noise-model-based validation to genuine hardware validation and let the fake-backend results in this paper be checked directly against reality.

Finally, this project's framing — calibration as noisy black-box optimization — deliberately set aside the more standard practice of fitting known functional forms (e.g., a Rabi-oscillation fit) directly. A worthwhile comparison for future work is between the black-box optimizers studied here and a shot-matched, fit-based calibration routine, to establish how much shot efficiency is

being left on the table by not exploiting the known functional form of $P(1) = \sin^2\left(\frac{\theta}{2}\right)$ — and whether that gap changes the practical calibration recommendation even if it doesn't change the relative ranking among black-box methods.

8. Conclusion

This paper comparatively analyzed four methods of recovering the rotation angle of a single-qubit RY gate to within 1° accuracy. According to the number of noisy measurement shots, the classical optimizers used substantially fewer shots than grid search when they declared success. SPSA converged most reliably (approx. 80% of 15 trials) at the highest per-success shot cost among the three classical optimizers, and Nelder-Mead the least reliable (approx. 27% of 15 trials) at the lowest per-success cost.

Nelder-Mead and COBYLA frequently terminate after declaring convergence while the recovered angle is still more than 1° from the true angle. Because it is invisible outside the simulation environment, this silent failure could cause critical errors. On the other hand, SPSA does not assure convergence due to its design and structure, so it may be more reliable and trusted when configured correctly for calibration pipelines.

This paper ultimately highlights the importance of considering silent convergence failures when architecting trustworthy calibration systems throughout single-parameter, simulation-based experiments, instead of claiming which optimizer is the best.

References

Aleksandrowicz, G., Alexander, T., Barkoutsos, P., Bello, L., Ben-Haim, Y., Bucher, D., et al. (2019). Qiskit: An Open-Source Framework for Quantum Computing. Zenodo. https://doi.org/10.5281/zenodo.2562111

Lavrijsen, W., Tudor, A., Müller, J., Iancu, C., & de Jong, W. (2020). Classical optimizers for noisy intermediate-scale quantum devices. arXiv preprint, arXiv:2004.03004.

Nelder, J. A., & Mead, R. (1965). A simplex method for function minimization. The Computer Journal, 7(4), 308–313. https://doi.org/10.1093/comjnl/7.4.308

Peruzzo, A., McClean, J., Shadbolt, P., Yung, M.-H., Zhou, X.-Q., Love, P. J., Aspuru-Guzik, A., & O'Brien, J. L. (2014). A variational eigenvalue solver on a photonic quantum processor. Nature Communications, 5, 4213. https://doi.org/10.1038/ncomms5213

Powell, M. J. D. (1994). A direct search optimization method that models the objective and constraint functions by linear interpolation. In S. Gomez & J.-P. Hennart (Eds.), Advances in Optimization and Numerical Analysis (pp. 51–67). Kluwer Academic. https://doi.org/10.1007/978-94-015-8330-5_4

Spall, J. C. (1992). Multivariate stochastic approximation using a simultaneous perturbation gradient approximation. IEEE Transactions on Automatic Control, 37(3), 332–341. https://doi.org/10.1109/9.119632

Virtanen, P., Gommers, R., Oliphant, T. E., Haberland, M., Reddy, T., Cournapeau, D., et al. (2020). SciPy 1.0: Fundamental algorithms for scientific computing in Python. Nature Methods, 17(3), 261–272. https://doi.org/10.1038/s41592-019-0686-2

국문 초록

단일 큐비트 게이트 보정을 위한 최적화 기법 비교: 샷 효율성과 조용한 수렴 실패 문제

은석준

국제트랙

용인한국외국어대학교부설고등학교

단일 큐비트 RY 게이트의 회전 각도 보정은 본질적으로 잡음이 있는 **블랙박스 최적화 문제**로 볼 수 있다. 보정자는 게이트의 실질적인 매개변수 값에 직접 접근할 수 없고, 오직 확률적 추정치만을 얻을 수 있다. 본 논문은 샷 노이즈가 있는 시뮬레이션 환경에서 알려지지 않은 회전 각도를 목표 정확도 이내로 복원하기 위한 네 가지 최적화 전략—그리드 탐색 (grid search), Nelder-Mead, COBYLA, SPSA—을 비교 평가하며, 샷 (shot) 수를 주요 비용 지표로 사용한다. 그리드 탐색을 제외한 이상적인 시뮬레이션 환경에서, SPSA 는 가장 높은 수렴률 (~80%)과 가장 높은 평균 비용을 보였다. Nelder-Mead 는 가장 낮은 수렴률 (~27%)을 보였으나, 수렴에 성공했을 때는 가장 낮은 평균 비용을 나타냈다. 그러나 본 연구의 핵심 결론이자 주장은, 샷 노이즈 하에서 Nelder-Mead 와 COBYLA 에서 빈번히 관찰되는 "조용한 수렴 실패 (silent convergence failure)"를 고려하는 것의 중요성과, 실제 참값 (ground truth)을 알 수 없는 실제 양자 하드웨어용 보정 파이프라인 설계에 있어 이것이 갖는 함의, 그리고 단일 매개변수·시뮬레이션 기반 연구라는 본 연구의 범위와 한계에 있다.

주요어: 양자 게이트 보정, 최적화 기법, 샷 노이즈, 샷 효율성, 조용한 수렴 실패 문제, 블랙박스 최적화.

학 번: 30109